SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1516

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments.* (BL3)

Dave's Taxonomy: Manipulation (Level 2) **Total Marks: 25**
Question Count: 5

Code of Conduct

I **R.JAIDEEP(192525441)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: R.JAIDEEP

Assessment Tasks
Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud.
   The system must handle millions of concurrent users with minimal buffering.
   Design a cloud architecture using scalability and caching techniques.
   Explain how load balancing and auto-scaling can ensure smooth streaming.
   Discuss cost optimization strategies for such high-demand workloads.

2. A fintech startup is deploying applications using containers across multiple cloud providers.
   They face challenges in maintaining consistency and managing deployments.
   Propose a solution using container orchestration and multi-cloud strategies.
   Explain how service discovery and scaling can be handled efficiently.
   Highlight the benefits and risks of a multi-cloud deployment model.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
   The organization needs a robust backup and disaster recovery strategy.
   Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
   Some applications must remain on-premises due to compliance requirements.
   Design a hybrid architecture that ensures secure communication between environments.
   Explain how identity management and encryption should be implemented.
   Discuss challenges in managing hybrid cloud environments.

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
   The system must continue operating even if one region goes down.
   Propose a multi-region deployment strategy in the cloud.
   Explain how data replication and failover mechanisms should be configured.
   Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	**Weightage**	**Level 4 - Exemplary**	**Level 3 - Proficient**	**Level 2 - Developing**	**Level 1 - Beginning**
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

1.A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques.Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

Cloud Architecture for a Scalable Video Streaming Platform

A media company serving **millions of concurrent users** needs an architecture that can handle sudden traffic increases while maintaining low latency and minimal buffering. A suitable cloud design should use **content delivery networks (CDNs), object storage, caching, load balancing, auto-scaling, and adaptive video streaming**. The main objective is to keep frequently requested video content close to users and dynamically increase computing capacity when demand rises.

1. Proposed Cloud Architecture

The architecture can be organized as follows:

Users → DNS → CDN/Edge Cache → Load Balancer → Auto-Scaling Streaming Servers → Object Storage

Supporting services such as databases, monitoring, authentication, and analytics can operate separately.

Users: Millions of viewers access the streaming platform through mobile applications, browsers, smart TVs, and other devices.

CDN and Edge Caching: The most important component for reducing buffering is a **Content Delivery Network (CDN)**. Video files are cached at geographically distributed edge locations. When users request popular videos, the CDN can deliver the content from a nearby edge server instead of repeatedly accessing the central cloud storage. This reduces latency and decreases the load on the origin infrastructure.

2. Scalability and Caching

Scalability should be implemented at multiple levels. **Horizontal scaling** can add more streaming servers when traffic increases instead of depending on one powerful server. The CDN provides the first level of scalability by serving cached video segments from edge locations.

A multi-level caching design can also be used:

- **CDN cache:** Stores frequently accessed video segments near users.
- **Application cache:** Stores frequently requested metadata such as video information and user preferences.
- **Database cache:** Reduces repeated queries to backend databases.
- **Origin storage:** Stores the permanent master and encoded video files.

3. Role of Load Balancing

A **load balancer** distributes incoming application requests across multiple streaming servers. Instead of allowing millions of users to connect to a single server, traffic is distributed among healthy instances.

For example, if ten streaming servers are available, the load balancer can distribute requests according to server capacity and current workload. **Health checks** can detect failed or overloaded servers and redirect new requests to healthy instances.

4. Role of Auto-Scaling

Auto-scaling automatically adjusts the number of cloud instances according to workload demand. During normal periods, only the required number of servers are maintained. When the number of concurrent viewers increases, additional instances are automatically launched.

For example:

Normal traffic → Few instances → Traffic increases → More instances → Traffic decreases → Instances removed

For major events such as sports finals or popular movie releases, **predictive or scheduled scaling** can be used to provision resources before the expected traffic spike.

5. Cost Optimization

Handling millions of users can become expensive, so cost optimization is important.

First, extensive use of CDN caching reduces the amount of data transferred from the origin infrastructure. This decreases origin server and storage access costs while improving performance.

Second, the company should use **auto-scaling** so that computing resources are not running unnecessarily during low-demand periods. Idle instances can be terminated automatically.

2.A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

Container Orchestration and Multi-Cloud Strategy for a Fintech Startup

A fintech startup deploying applications across multiple cloud providers must maintain **consistency, security, availability, and efficient deployment**. Different cloud platforms may have different networking, storage, and deployment configurations, which can make application management difficult. A suitable solution is to use **container orchestration with Kubernetes**, supported by Infrastructure as Code (IaC), centralized CI/CD, service discovery, and automated scaling. This approach allows the same containerized applications to run consistently across multiple cloud environments.

1. Proposed Container Orchestration Solution

The startup can package each application as a **container image** containing the application code, libraries, dependencies, and configuration requirements. These images can be stored in a common container registry and deployed using **Kubernetes clusters** running across different cloud providers.

A simplified architecture is:

Developers → CI/CD Pipeline → Container Registry → Kubernetes Clusters → Cloud Provider A / Cloud Provider B / Cloud Provider C

Kubernetes provides a common orchestration layer for deploying and managing containers. The same Kubernetes manifests or Helm charts can be used across different cloud environments, reducing configuration differences and improving consistency.

Infrastructure as Code (IaC) tools such as Terraform can define networking, clusters, security rules, and other infrastructure components as reusable configuration. This reduces manual configuration and helps ensure that environments remain consistent.

2. Deployment and Consistency

A centralized **CI/CD pipeline** can automatically build, test, scan, and deploy container images. When developers release a new version, the pipeline creates a versioned container image and pushes it to the container registry. The same tested image can then be deployed across multiple Kubernetes clusters.

Configuration should be separated from application code using Kubernetes **ConfigMaps and Secrets**. Sensitive fintech information such as database credentials and API keys should be stored securely and should not be embedded directly inside container images.

Deployment strategies such as **rolling updates, blue-green deployment, and canary deployment** can reduce downtime and deployment risks. A new version can initially be deployed to a small percentage of users before being expanded to the entire environment.

3. Service Discovery

In a microservices-based fintech application, services such as authentication, payment processing, transaction management, and notification services need to communicate with one another.

Kubernetes provides built-in **service discovery** using Services and DNS. Instead of connecting directly to changing container IP addresses, applications communicate using stable service names.

For example:

```
Payment Service → Authentication Service
        → Transaction Service
        → Notification Service
```

If a container is restarted or moved to another node, its IP address may change, but the Kubernetes Service continues to provide a stable endpoint.

4. Efficient Scaling

Kubernetes supports automatic scaling at multiple levels.

Horizontal Pod Autoscaler (HPA) can increase or decrease the number of application pods based on CPU, memory, request rate, or other application metrics. For example, if payment-processing traffic suddenly increases, Kubernetes can automatically create additional payment-service pods.

Cluster Autoscaler can add or remove worker nodes when existing nodes do not have enough capacity. This provides another layer of scalability.

For predictable events, such as a major financial transaction period, scheduled scaling can provision additional capacity in advance.

A typical scaling process is:

Increased traffic → HPA detects demand → New pods created → Cluster adds nodes if required → Load balancing distributes requests

This allows the system to respond dynamically without requiring administrators to manually provision servers.

5. Benefits of Multi-Cloud Deployment

High Availability

Running applications across multiple cloud providers reduces dependence on a single provider. If one cloud experiences a major outage, workloads can potentially be shifted to another provider.

Reduced Vendor Lock-In

Using Kubernetes and portable container images makes it easier to move applications between cloud platforms. The startup is therefore less dependent on proprietary services from one provider.

Geographic Performance

Different cloud providers may have stronger infrastructure in different regions. Workloads can be deployed

6. Risks and Challenges

Multi-cloud deployment also introduces significant complexity.

Management complexity: Multiple cloud accounts, Kubernetes clusters, networks, monitoring systems, and security policies must be managed consistently.

Security risks: Different cloud providers may have different identity, networking, and security mechanisms. Misconfiguration can expose sensitive financial information.

Data consistency: Financial transactions require strong consistency. Replicating transaction data across clouds can introduce synchronization delays or conflicting data if not designed carefully.

Higher operational cost: Running infrastructure across multiple clouds may increase networking, monitoring, support, and administrative costs.

Network latency and data-transfer charges: Moving data between cloud providers can increase latency and generate additional network-transfer cost

3.Cloud Backup and Disaster Recovery Strategy for an E-Learning Platform

An e-learning platform stores important data such as **student profiles, course materials, assignments, examination results, attendance records, and payment information**. Accidental deletion can cause serious data loss and interrupt online learning. A robust disaster recovery (DR) strategy should therefore combine **cloud-native backup, data replication, versioning, automated recovery, and regular testing**.

1. Cloud-Native Backup and Versioning

The organization should use cloud backup services to automatically back up databases, application configurations, and important files. Course videos, documents, and other objects should be stored in cloud object storage with **versioning enabled**. Versioning maintains previous copies of files, so if an instructor or administrator accidentally deletes or overwrites a file, an earlier version can be restored.

Database backups should include **regular full backups and frequent incremental or transaction-log backups**. Backup retention policies should keep multiple recovery points for an appropriate period. Backups should also be encrypted and protected using access controls so that unauthorized users cannot delete or modify them.

2. Data Replication

Critical databases should be replicated to a **secondary availability zone or geographically separate cloud region**. Replication ensures that a current copy of important data is available if the primary infrastructure fails.

For example:

Primary Region → Continuous Replication → Secondary Region

If the primary region becomes unavailable, the secondary environment can be activated. Critical application servers, databases, and storage should be replicated according to their importance. Cross-region replication also protects against regional disasters rather than relying only on backups within the same datacenter.

3. Achieving RPO and RTO

Recovery Point Objective (RPO) defines the maximum acceptable amount of data that can be lost after a failure. For example, an RPO of **15 minutes** means the organization should be able to recover data to a point no more than 15 minutes before the failure.

Recovery Time Objective (RTO) defines the maximum acceptable time required to restore the service after a failure. For example, an RTO of **30 minutes** means the platform should be operational within 30 minutes.

RTO can be achieved through:

- Preconfigured disaster recovery infrastructure.
- Automated deployment and recovery scripts.
- Replicated databases and storage.
- Automated DNS or traffic redirection.
- Regular recovery testing.

Thus, **RPO focuses on how much data can be lost, while RTO focuses on how quickly the service must be restored.**

4. Business Continuity During Failures

1. **Detect the failure:** Monitoring systems should immediately identify data loss, application failure, or infrastructure problems and generate alerts.
2. **Stop further damage:** If accidental deletion is detected, affected accounts or processes should be restricted to prevent additional data from being deleted or overwritten.
3. **Identify the recovery point:** Administrators should select the most recent valid backup or object version based on the required RPO.
4. **Restore data:** Deleted files can be recovered using object-storage versioning, while databases can be restored from backups or replicated copies.
5. **Activate the secondary environment:** If the primary region is unavailable, traffic can be redirected to the secondary region containing replicated application and database resources.
6. **Verify the restored system:** Data integrity, user accounts, course materials, and application functionality should be tested before declaring the service fully operational.
7. **Return to normal operations:** After the primary environment is repaired, data should be synchronized and workloads can be safely moved back.

4.Hybrid Cloud Architecture for a Government Agency

A government agency often needs to process both **sensitive and non-sensitive data**. Sensitive applications may have to remain on-premises because of compliance, data-sovereignty, or security requirements, while non-sensitive applications can benefit from the scalability and flexibility of public cloud services. A **hybrid cloud architecture** combines private/on-premises infrastructure with public cloud resources while providing secure communication and centralized security controls.

1. Proposed Hybrid Cloud Architecture

Sensitive applications such as citizen records, confidential government databases, and regulated workloads remain in the **on-premises/private environment**. Non-sensitive applications such as public websites, information portals, and general analytics can be hosted in the public cloud.

A secure **site-to-site VPN or dedicated private connection** should connect the two environments. Firewalls, intrusion detection systems, and network segmentation should control traffic between the on-premises and cloud environments. Only required application-to-application communication should be permitted.

2. Secure Communication

Communication between the two environments should use **encrypted network connections**. A site-to-site VPN using strong encryption can establish a secure tunnel between the government datacenter and cloud Virtual Private Cloud (VPC)/Virtual Network.

For higher-performance and predictable connectivity, a **dedicated private connection** can be used instead of relying entirely on the public internet. Firewalls should restrict traffic to approved IP addresses, ports, and services.

Application-level communication should additionally use **TLS/HTTPS**, ensuring that sensitive information remains encrypted while travelling between the on-premises and cloud environments.

Network segmentation should separate sensitive systems from public-facing applications. A compromised public-cloud application should not have unrestricted access to internal government databases.

3. Identity Management

A centralized **Identity and Access Management (IAM)** system should be used to control access across both environments. The agency can use a central identity provider and integrate on-premises directory services with cloud IAM.

Users should receive access according to the **least-privilege principle**, meaning they receive only the permissions required for their jobs. For example, a government employee working with public information should not automatically receive access to confidential citizen databases.

Multi-factor authentication (MFA) should be mandatory for administrators and users accessing sensitive applications. **Role-Based Access Control (RBAC)** can assign permissions based on roles such as administrator, analyst, officer, or auditor.

Identity federation can allow authorized users to use a common organizational identity across on-premises and cloud services while maintaining centralized authentication and auditing.

4. Encryption

Encryption should be implemented for both **data in transit and data at rest**.

Data in transit:
All communication between users, cloud services, and on-premises applications should use secure protocols such as TLS. VPN or dedicated encrypted connections should protect communication between the two environments.

Data at rest:
Sensitive databases, files, backups, and storage volumes should be encrypted. Encryption keys should be managed through a secure **Key Management Service (KMS)** or hardware security module (HSM). Access to encryption keys should be strictly controlled and logged.

Backups should also be encrypted because backup copies may contain the same sensitive information as production systems.

5. Challenges of Hybrid Cloud Management

Security complexity: Managing security across on-premises and public-cloud infrastructure is more difficult because both environments have different security tools, configurations, and policies.

Compliance: Government agencies must ensure that data remains in approved locations and follows applicable regulatory and data-sovereignty requirements.

Identity management: Maintaining synchronized identities, roles, and permissions across multiple environments can become complicated.

Network complexity: VPNs, firewalls, routing, DNS, and connectivity between environments require careful configuration. Network failures can affect applications that depend on both environments.

Monitoring: Logs and security events are generated by servers, cloud services, applications, and network devices. Centralized monitoring is required to detect threats and troubleshoot problems.

Cost management: Running on-premises infrastructure while also paying for cloud resources, network connectivity, storage, and security services can increase operational costs.

Skill requirements: IT teams need expertise in traditional infrastructure as well as cloud platforms, virtualization, networking, security, and automation.

5,Multi-Region Cloud Deployment for High Availability and Fault Tolerance

A SaaS provider serving many customers must ensure that its application remains available even when a complete cloud region experiences an outage. A **multi-region deployment** provides high availability by running application components in two or more geographically separate cloud regions. If one region fails, users can automatically be redirected to a healthy region. The architecture should combine **multi-region application deployment, data replication, automated failover, load balancing, and continuous monitoring**.

1. Proposed Multi-Region Architecture

The SaaS application should be deployed in **at least two geographically separated regions**. Each region should contain load balancers, application servers or containers, databases, storage, and required supporting services.

Users should connect through a **global load balancer or DNS-based traffic management system**. Under normal conditions, traffic can be distributed between both regions. This is preferable to keeping the second region completely unused because it allows both regions to handle production workloads.

2. Data Replication

Data replication is essential because simply duplicating application servers does not protect the SaaS provider from data loss.

For databases, **cross-region replication** should be configured. Depending on the application's consistency requirements, synchronous or asynchronous replication can be selected.

- **Synchronous replication** keeps copies highly consistent but may introduce additional latency between regions.
- **Asynchronous replication** provides better geographic performance but may result in a small amount of data loss if the primary region fails before the latest changes are replicated.

Critical data should therefore use a replication method that matches the required **Recovery Point Objective (RPO)**.

Object storage such as documents, images, and application files should also use **cross-region replication**. Backups should be stored separately from the production environment to provide additional protection against accidental deletion or corruption.

3. Failover Mechanism

The system should use **automated health checks** to continuously verify the availability of each region. These checks should test not only whether servers are running but also whether important application functions and databases are responding correctly.

If Region A becomes unavailable:

Failure detected → Health check fails → Traffic removed from Region A → Requests redirected to Region B → Region B continues serving users

A global traffic manager can automatically stop routing new requests to the failed region. DNS records or traffic-routing policies can then direct users to the healthy region.

For faster recovery, Region B should already have sufficient infrastructure available. The application servers can be maintained in an active-active or active-passive configuration depending on business requirements.

4. Active-Active vs. Active-Passive

In an **active-active** architecture, both regions serve users simultaneously. This provides better resource utilization and faster failover because the second region is already processing traffic.

In an **active-passive** architecture, one region normally serves users while the second region remains ready as a standby environment. This can be simpler but may result in slower failover and unused resources.

For a large SaaS provider, **active-active deployment is often preferred** when the application and database architecture can support multi-region consistency.

5. Monitoring and Alerting

Continuous monitoring is essential for detecting problems before they become major outages. The provider should monitor:

- CPU and memory utilization.
- Application response time and latency.
- Error rates and failed requests.
- Database health and replication lag.
- Network connectivity.
- Load balancer health.
- Storage availability.
- Number of active users and requests.
- Regional availability.

A centralized monitoring platform should collect **metrics, logs, and traces** from both regions. Dashboards can display the health of the complete SaaS environment.

Alerts should be categorized according to severity. **Critical alerts** should immediately notify the operations team through appropriate channels, while lower-priority warnings can be reviewed during normal operations.

6. Proactive Issue Detection

The provider should not wait for a complete regional failure before testing its DR system. Regular **failover and disaster-recovery tests** should be performed. Controlled failure simulations can verify whether traffic is correctly redirected and whether replicated data is available.